Do Monitors of Latent Thoughts Read What the Model Uses?

A causal audit of continuous-thought reasoners, and of a published latent safety monitor

Muhammad Jon Raza*
Department of Electrical Engineering
Lahore University of Management Sciences
jon@titannetwork.com

Muhammad Tahir
Department of Electrical Engineering
Lahore University of Management Sciences
tahir@lums.edu.pk

Abstract

Several recent language models reason in a short sequence of continuous vectors ("latent thoughts") rather than in text, and the proposed way to oversee them is a linear probe on those vectors. A probe establishes what a thought *contains*. It does not establish that the model *uses* what it contains. We ask whether the two come apart, and we answer with interventions only, on released checkpoints, with every prediction registered before each run. First, a one-command audit measures for each thought whether it carries a value into the answer, whether the model needs it, and whether a probe's reading of it is confirmed by behaviour. On eight checkpoints (CODI and SIM-CoT at 124M to 8B, Coconut, and two models trained on a logic task) the thoughts that read best are routinely the ones the model never acts on; below 8B the readable slot holds a stale copy of a value an earlier thought computed and handed on. Second, we retrain the model of a published latent safety monitor (Ramjee et al., 2026) from its authors' recipe, together with an inert-trigger control the original lacked. The probe transfers at 1.000 on the backdoored model and 0.53 on the control, yet swapping, removing or editing the probed thoughts moves the model's behaviour by less than 0.005 against a 0.19 effect: the probe reads a training-installed copy of the trigger, and the backdoor is decided in the memory of the ordinary prompt words. Third, a model retrained so that its answer can only read its thoughts makes them load-bearing, and there steering along the direction the behaviour moves on removes the backdoor's effect, while steering along the direction the probe reads moves nothing on either model. Two legitimate uses of a thought monitor survive: a tamper on the carrying thought leaves a seam the next position does not follow (AUROC 0.92 to 0.98), and routing the least confident quarter of problems to the model's own written solution gains 3.7, 7.5 and 10.2 points at 1B, 3B and 8B. Nothing read from the thoughts improves either. All 39 runs reproduce byte for byte.

1 Introduction

A model that reasons in text can be read; its steps are tokens. Coconut (Hao et al., 2024), CODI (Shen et al., 2025) and SIM-CoT (Wei et al., 2026) move the steps into a handful of continuous vectors that no reader can decode directly, and the field's response has been to train a probe on the vectors and treat its accuracy as monitorability. Ramjee et al. (2026) go furthest: they plant a dual-trigger backdoor in a Coconut-style model, show that a probe on the early thoughts detects the armed state at 89%, and recommend monitoring those positions.

*Roll number 27100293.

Preprint. Under review.

That recommendation rests on an assumption the probe cannot test. A probe shows that a quantity is linearly present in a vector. Whether the model goes on to use that quantity, and whether the answer would differ without it, are causal questions, and prior work on these models has raised them without settling them on released weights. Cywiński et al. (2025) patched single thoughts of a self-trained CODI on templated problems; Dilgren and Wiegreffe (2026) showed that on logic tasks the thoughts can be dropped at no cost; Aswal et al. (2026) showed that decodable patterns appear in controls without recurrence; Li et al. (2026) measured flip rates under step-wise zeroing. None asked, for a given thought, whether what a probe reads from it is what the model acts on, and none applied that test to a monitor that someone has proposed to deploy.

This paper narrows the question to one distinction, *readable* against *used*, and decides it three ways. We run no training on any released model; we change what is inside the model while it runs and watch the answer. Every run was preregistered in a public ledger with its predictions, scored afterwards, and reproduced on a second account.

The closing tests are these. (i) On eight released checkpoints, does a thought that a probe reads well carry a value into the answer, and does the model need it? We find that one or two of six do, that the model often does not need even those, and that the most readable thought is frequently a stale copy of a value handed on earlier. (ii) On the model of a published safety monitor, retrained from its authors' recipe with a control they did not run, does the probe read the cause of the behaviour or a copy of its trigger? We find a copy: perfect transfer, no causal effect, and the behaviour decided in the prompt's own memory. (iii) Can the thoughts be steered, and does that depend on whether the model uses them? We find that it depends on exactly that, by building a model whose answer can read only its thoughts and repeating the audit on it. Two uses of a thought monitor remain, tamper evidence and confidence routing, and we measure both.

2 Related work

Continuous-thought models. Coconut feeds the last hidden state back as the next input embedding and replaces reasoning steps by a curriculum. CODI distils an explicit-chain teacher into six thoughts through a projector. SIM-CoT adds step-level supervision. We audit released CODI and SIM-CoT checkpoints, a third-party Coconut retrain, and the ProsQA checkpoints of Dilgren and Wiegreffe (2026).

Causal analyses of latent thoughts. Cywiński et al. (2025) found, on twenty three-step templates, that the third and fifth latents of CODI-1B store the two step values and that the last two can be replaced at no cost; our carriers and inert terminal confirm that placement on natural data. Li et al. (2026) zero single steps and report flip rates; Dilgren and Wiegreffe (2026) mean-ablate all latents and decode intermediate results from the top-1 integer token; Jin et al. (2026) find accuracy and counterfactual responsiveness diverging over training. None separates a thought's transport from its necessity, nor a probe's reading from its confirmation.

Editing thoughts. Chang et al. (2026) edit an early cached thought at decoding time and transplant latent sequences between instances, gaining 0.4 to 1.8 points without a value-matched donor or a control. Huang and Qi (2026) find that synthetic steering does not reach the output. Our write is value-matched, controlled, and reported with its harm and its ceiling.

Monitors. Ramjee et al. (2026) is the monitor we test. That decodability is not causality is established in principle (Open Problems, 2025; Decodability, 2026); we measure the gap per thought on latent reasoners and on a proposed monitor.

3 Instruments

Two routes. A thought at slot s reaches the answer by its output state, projected into slot $s{+}1$'s input, and by the keys and values it writes into the cache that every later position attends to. Hooks edit either route on the released decoding path; feeding a thought its own recorded state or cache back is the identity, which is checked before every run.

Twins and transport. Swapping an unrelated problem's thought breaks answers, and breaking is not carrying. A *twin* changes one number in the question and recomputes the chain, so the pair differ in one intermediate value and in the answer. Only pairs the model solves both ways are kept. Swapping one site between twins, in both directions, and recording whether the answer moves to the twin's value gives *transport*; whether it merely changes gives *disturbance*. Chains of at most two steps are *short*, three or more *long*.

Masking and necessity. Each thought is removed two ways, by replacing its state with the pool mean and by hiding its cache entry from all later positions; the larger accuracy drop is its cost, with a row bootstrap.

Readability and faithfulness. A probe (ridge and a two-layer MLP) reads the penultimate intermediate value from each thought's state. Its *readability* is held-out accuracy. Its *faithfulness* is the share of its alarms on twin pairs that the behaviour confirms: the probe reads a different value in the two twins and the swap of that thought moves the answer. A slot is a *carrier* if it transports ≥ 0.15 on long chains, *inert* if ≤ 0.05, and *readable but unfaithful* if read at ≥ 0.30 and confirmed ≤ 0.05.

The card. One command produces, per model, dataset and GPU session, a card with these quantities per slot, fixed thresholds, five checks, a regime (*critical path* if a carrier costs ≥ 10 points, *backup route* if it carries but costs less, *no transport* if nothing carries) and a monitor-placement verdict, with fingerprints that let two sessions be compared bit for bit.

Preregistration. Each experiment is a numbered ledger section with its predictions written before the run and scored after. Wrong predictions are kept. Checkpoints are pinned by revision and SHA-256.

4 What the thoughts do on released checkpoints

Table 1 lists the checkpoints; each reproduces its paper's figure within three points. Pools are GSM8K-Aug training rows, the GSM8K test and train sets with their calculator annotations as chains, ASDiv, SVAMP as a one-step control, and ProsQA.

Table 1: Checkpoints audited as released. Accuracies are our greedy reproductions.

Model	Base	Paper	Ours
CODI GPT-2	GPT-2 124M	43.7	43.6
CODI-1B	LLaMA-3.2-1B-Instruct	55.6	55.5
SIM-CoT-1B	LLaMA-3.2-1B-Instruct	56.1	55.7
SIM-CoT-3B	LLaMA-3.2-3B-Instruct	62.3	61.9
SIM-CoT-8B	LLaMA-3.1-8B-Instruct	64.1	62.5
Coconut (retrain)	GPT-2 124M	34.6	34.6
Coconut, ProsQA	LLaMA-3.2-1B-Instruct	97.6	97.6
CODI, ProsQA	LLaMA-3.2-1B-Instruct	99.0	99.0

4.1 Used, and not needed

Swapping the whole cache of thoughts moves 60 to 75% of answers on every CODI-family model, yet on long chains a single thought transports the value only at one or two slots: slot 3 at CODI-1B (0.30 to 0.35), slots 1 and 3 at GPT-2 and SIM-CoT-1B (0.15 to 0.36), and the cache of the position after each at the same rate. The carrier computes and the next position's memory stores. Below 3B no single site transports more than 0.10 on short chains: a chain-length gate. The carriers are sufficient and, at 1B, not necessary: removing CODI-1B's carrier costs 2 to 4 points with upper bounds under 10 on every pool, while hiding position 0's memory of the question costs 19 to 59 points at GPT-2 and SIM-CoT-1B. We call this the *backup route* regime, and it holds at every scale we ran.

4.2 Readable is not used, and why

On every card up to 3B there is a slot that a probe reads at 0.4 to 0.8 and the behaviour confirms at most 5% of the time, under both probe families: the last thought at 1B and below, the store at 3B. Carrier faithfulness moves by at most 0.04 between probe families, so faithfulness belongs to the slot and not to the probe. Probing the same states for what the unfaithful slot holds gives the mechanism. The question's attributes are spread evenly over every slot at a modest level, but the computed penultimate value is read from CODI-1B's inert last thought at 0.47, above the carrier's 0.42, and from the 3B store at 0.42 against 0.46 for the carrier. The unfaithful slot holds a copy of the value the carrier computed, carried forward in the residual stream after the carrier handed it to the cache, as readable as at the source and no longer used. Readability outlives use, and a probe placed by readability lands on the copy. Probes also read the question's first operator and step count from any slot, including the unused one, at 0.7 to 0.9, so a monitor can score well while reading the prompt.

4.3 Scale moves the map

Table 2 follows SIM-CoT from 1B to 8B. The carriers move to the even slots at 3B and split into two pairs at 8B, one for short chains and one for long, so the gate reverses rather than vanishes. The question register falls from 33 to 59 points of necessity to 4.6. At 8B a thought is the most necessary single site for the first time, and the readable-but-unfaithful slot is gone: readability correlates with faithfulness at 0.71 to 0.75. The larger the model, the more the thoughts do the work and the more honestly a probe on them reads. The stale copy is a property of the small checkpoints; the backup route and the one-or-two-slot computation are not.

Table 2: SIM-CoT across scale, GSM8K test, long chains. Costs in accuracy points.

	1B	3B	8B
Carriers (state transport)	1, 3 (0.22, 0.20)	2, 4 (0.27, 0.16)	2 long (0.28); 4 short (0.51)
Store (K/V transport)	2, 4	3 (0.28)	3 long (0.29); 5 short (0.51)
Short-chain single-site transport, max	0.05	0.23	0.51
Terminal slot	inert	K/V in use (0.15)	K/V in use (0.51 short)
Carrier cost / most necessary site	4–10 / register, 32.5	6.5, 8.9 / register, 10.8	4.6 / thought 4, 13.3
Readable-but-unfaithful slot (read / confirmed)	slot 5 (0.32 / 0.00)	slot 3 (0.55 / 0.02)	none; Spearman 0.71
Regime	backup route	backup route	backup route

4.4 Two contrasts

The Coconut retrain inverts the profile: its thoughts carry on one-step problems, every pair is necessary (45 to 49 points), its last position is in use, and a value cannot be written into it. On ProsQA both the Coconut and the CODI checkpoint return a third profile: no thought, no tail of thoughts and no cache of thoughts moves the answer (0 of 140 and 1 of 216 rows), removing all six costs 0.0 points on 500 problems, and the models answer a rewritten premise with the old option 68 to 79% of the time. There is nothing in those thoughts to monitor. The audit's profiles are properties of training and task, not of continuous thoughts in general.

5 A published monitor, tested causally

Ramjee et al. (2026) train a Coconut-style GPT-2 on MoralChain, 12,000 social scenarios with a moral and an immoral path, with a dual-trigger backdoor: `[T]` arms immoral latent reasoning but keeps the moral action, `[O]` alone is a control, `[T][O]` releases the immoral action. A probe trained on `[T][O]` against `[O]` transfers to `[T]` against baseline at 89% at the first latent and 58% at the sixth, and the paper recommends monitoring the early latents. No weights were released and nothing in the paper tests whether the probed vectors are used.

Setup. We trained two models with their released code and recipe (GPT-2 124M, five curriculum stages, two latents per step; one deviation, an optimiser batch of 32 reached by accumulation on a 15 GB GPU). Model A is their backdoor. Model B is an *inert-trigger control*: the same prompts and trigger tokens in the same proportions, with the moral path under every condition, so the tokens are

present and nothing is attached to them. On each we ran their probe protocol, a behavioural gate, and the interchange arms of Section 3 on the 1,000 test scenarios. Because the trained model writes a new action rather than copying a reference, their word-overlap judge cannot score it; an 8B instruction model chooses between the two reference actions, validated at 0.99 and 0.93 on paraphrases it never sees as candidates.

Table 3: Their probe protocol on both models: transfer accuracy per latent position, 1,000 scenarios.

	z_1	z_2	z_3	z_4	z_5	z_6
Model A, backdoor	**1.000**	1.000	1.000	1.000	1.000	1.000
Model B, inert trigger	0.528	0.515	0.487	0.464	0.476	0.475
Reported in Ramjee et al. (2026)	0.894	0.847	0.783	0.712	0.648	0.583

Table 4: Model A: share of immoral actions under each condition and under the interchange arms (8B judge, 95% bootstrap). Every arm sits on its clean reference.

Run	Immoral share	Reference
Baseline / armed / control / release	0.163 / 0.191 / 0.164 / **0.352**	paper: 0.06 / 0.08 / 0.06 / 0.88
Control run given the armed run's six latents	0.164 [0.141, 0.186]	control 0.164
Control run given the armed run's z_1 only, or z_1 to z_3	0.159 / 0.162	control 0.164
Release run given the baseline run's latents	0.347 [0.319, 0.376]	release 0.352
Release run with every latent hidden from the decode	0.388 [0.359, 0.420]	release 0.352
Release run with the probe direction removed from every latent	0.354	release 0.352
Control run shifted along the probe direction by the class gap	0.164	control 0.164
Release run with every context word's cache entry from the control prompt	**0.183** [0.16, 0.21]	control 0.164
Release run with `[T]`'s own cache entry replaced for the answer	0.361 [0.33, 0.39]	release 0.352
Native: a leading space in place of `[T]`, no trigger token present	**0.321**	release 0.352

The probe reads a copy. On our backdoored model the probe does not merely transfer at 89%; it separates armed from baseline perfectly at all six latents, so there is no early-to-late decay to monitor. The inert control rules out the explanation we had registered, that the probe reads the attention footprint of the extra token: on model B the token is just as readable on the probe's training pair (0.88 to 0.98) and the transfer is at chance. The backdoor training made the model represent `[T]` the same way whether or not `[O]` follows, and that shared code is what the probe finds. The code does nothing. Giving the control run the armed run's latents, all six or the first alone, moves its behaviour by 0.000; replacing the release run's latents with the baseline's, hiding them, or deleting the probe's own direction from them moves the release by at most 0.005 against an effect of 0.188, and hiding them raises it. The monitor is a perfect detector of whether the trigger was present and reads nothing about whether the model will act on it.

Where the backdoor lives. Following the behaviour with cache substitution, the answer tokens never read `[T]`'s own cache entry; `[O]`'s entry carries a quarter to a third of the release, agreed by three instruments; and replacing the cache entries of the ordinary scenario words with the entries the same words produce under the control prompt brings the release to within 0.02 of the control, with two donors agreeing to the third decimal. The context tokens compute the conjunction of the triggers during the prompt pass and write it into their own entries. The backdoor is also looser than a trigger-based evaluation assumes: a single leading space followed by `[O]` releases at 0.321 against 0.352 for the real pair. A perfectly readable "this token saw `[T]`" direction in `[O]`'s residual is inert too; deleting it from eleven blocks changes nothing, and a random direction gives the same number.

What is weaker than in their paper. Our release reaches 0.35 against their 0.88 and our baseline 0.16 against 0.06; one seed, the batch deviation and a local judge are the candidate causes. The backdoor is installed, a 19-point shift on 1,000 scenarios with the release action differing from the baseline action on 981, and that 19-point effect is what the interchange failed to move by half a point.

6 When thoughts can be steered

If the thoughts hold a copy, aligning them cannot change behaviour, and the constructive question is whether a model can be built whose thoughts are used, what it costs, and whether aligning them then works. We trained a *bottleneck* model: model A's data and recipe with one change to the attention mask, so that the end-of-latent marker and every token after it cannot read the prompt. Everything the action needs must pass through the six latents. On both models we steered the thoughts along two directions, the probe's own direction by one to four class gaps and the class mean difference (mean release latent minus mean control latent) by one and two lengths, and measured the probe's reading, the generated text and the action together (Table 5).

Table 5: The same audit on the unrestricted model A and the bottleneck model. Shares are immoral actions on 1,000 scenarios; differences are paired with 95% intervals.

	Unrestricted	Bottleneck
Baseline moral share; what the model writes	0.84; two reasoning steps, then the action	0.82; the action line only
Release − control	0.188	0.089 [0.062, 0.117]
Probe transfer, $z_1 \ldots z_6$	1.000 at every latent	0.83, 0.65, 0.63, 0.60, 0.58, 0.56
Release given the baseline's latents	$+0.005$	-0.046 [-0.069, -0.022]
Steer along the probe direction, 1 to 4 gaps	0.000; 99–100% of texts identical	0.000; 100% identical
Probe gap as a share of the latent norm	0.06–0.09%	0.01–0.05%
Steer along the class mean difference, $\times 2$, release	$+0.002$ [-0.012, $+0.016$]	$\mathbf{-0.080}$ [-0.108, -0.052]
Mean difference as a share of the norm; cosine to the probe direction	9–18%; 0.005–0.007	6–12%; 0.001–0.004
Text damage under the mean-difference steer	none; 36–88% of texts reworded, verdicts unmoved	none

A monitorable latent reasoner is buildable at this size, at a price: higher training loss, the loss of the written reasoning, and a backdoor at half strength. Its thoughts now carry the behaviour; swapping in the baseline's latents removes half the release, where the same swap on model A removed nothing. Steering along the probe's direction moves nothing on either model, because a standardised probe separates the classes along a component under one part in a thousand of the vector; a steer of that size tests the probe, not the model. Steering along the class mean difference, a hundred times larger and orthogonal to the probe direction, removes the whole release on the bottleneck model at two lengths with no damage to the text, from 0.278 to 0.198 against a control of 0.189. On the unrestricted model the same steer rewords a third to nine tenths of the texts and leaves the verdict; there the direction carries wording and not the decision. Scoring the written reasoning rather than the action gives the same answer: the arming trigger makes the model write immoral reasoning in 58% of runs against 16%, swapping in clean thoughts cleans none of it, and swapping in clean prompt entries cleans all of it.

7 What a monitor on the thoughts can do

Tamper evidence. The audit's map says where a write would take: the carrier. We wrote, into correctly solved problems, a natural thought whose value implies a wrong answer chosen in advance, on CODI-1B, SIM-CoT-3B and SIM-CoT-8B. One vector moves 13 to 21% of correct answers to the attacker's value against 0.2 to 1.5% for a donor carrying some other value, and writing the victim's own value back leaves 91 to 94% intact. We predicted that the later thoughts would copy the written value. They did not, at any size: after a hijack the slot the carrier feeds still reads the original value on 35 to 57% of cases and the attacker's on at most 13%. The stale copy that makes a probe unfaithful in Section 4 is, under attack, a seam, and a linear detector on that one slot tells hijacked from clean at 0.92, 0.97 and 0.98 AUROC at 1B, 3B and 8B, where the written slot itself sits at 0.68 to 0.76. The same detector reads the model's own natural slips at 0.65 to 0.86: a foreign value is a foreign body, and the model's own error is a near miss. That gap is the specificity a tamper-evidence claim needs.

Accuracy. The value-matched write also repairs failures, 14 to 23% of them against a 2 to 4% control up to 3B, but its ceiling is the failure mix: 40 to 68% of failures are plan errors with no value to write, and at 8B the write repairs latent slips at 11% against an 8.5% random control. What rises with scale is the model's own written solution, right on 20%, 39% and 53% of the latent failures at 1B, 3B and 8B. Routing the least confident quarter of problems to it, on the answer's first-token probability alone, gains 3.7, 7.5 and 10.2 points at 1B, 3B and 8B, and half routed beats or matches running the written solution everywhere. Nothing read from the thoughts improves the gate: a fitted

combination of confidence with the six states is worse at a quarter routed on three of four models, and the thoughts' own consistency, which sees a natural slip faintly, ties confidence at best.

8 Discussion and limitations

What the three tests agree on. A probe on a latent thought measures presence, and presence outlives use. On released CODI-family models the readable slot is a stale copy; on a proposed safety monitor it is a copy of the trigger; on a model built so that its thoughts are the only channel the gap closes, and steering works exactly there and nowhere else. The practical rule follows: before trusting a probe on a thought, run the interchange that would move the behaviour if the thought carried it, and include a prompt-only control. Content that no interchange moves is a copy, and a safety claim about a copy is a claim about the detector.

Reach of the claims. The released-checkpoint results hold for the CODI family up to 8B on grade-school arithmetic, with ProsQA as a null and Coconut as an inversion; the chain-length gate and the readable-but-unfaithful slot are properties of the checkpoints below 8B. The monitor results hold for a 124M model trained from the authors' recipe, one seed per model, with a weaker release rate than theirs (0.35 against 0.88) and a local 8B judge in place of GPT-4o. A second seed at their batch size, a frontier-model calibration of the judge on a few hundred items, and one backdoor at a larger scale are the checks that would extend them. The 8B audit ran in float16 on two T4s and scored 1.6 points under its paper. The routing cost is counted in problems, not tokens. About a third of our registered predictions were wrong; each is recorded with the number that refuted it.

9 Conclusion

Readable is not used. On eight released latent reasoners the thoughts a probe reads best are often copies the model has finished with, and a published latent safety monitor reads such a copy of its trigger while the behaviour it is meant to catch is decided elsewhere. The audit that shows this costs about one GPU-hour per model, needs no training, and tells a practitioner which slot, if any, is live, whether a tamper there would be visible, and what the thoughts can add to a router, which on these models is nothing beyond the model's own confidence.

Reproducibility

The repository `github.com/0x0shephard/latent-reasoning` contains the audit driver, every runner, the pinned Kaggle notebooks, 16 cards, the trained models' recipes and outputs, and the ledger (`docs/RESEARCH_CONTEXT_LEDGER.md`, sections 108 to 149) with each preregistration and its scoring. Checkpoints are pinned by revision and SHA-256; each card records the GPU, library versions and dataset hash; reruns on a second account reproduced the twin predictions byte for byte.

References

D. Aswal, T. Palmeira Ferraz, Y. Zhou, and M. Peyrard. Observable patterns are not explanations: A causal-geometric analysis of latent reasoning models. *arXiv:2606.12689*, 2026.

S. Chang et al. Unlocking the black box of latent reasoning: An interpretability-guided approach to intervention. In *ACL*, 2026. arXiv:2606.01243.

B. Cywiński, B. Bussmann, A. Conmy, J. Engels, N. Nanda, and S. Rajamanoharan. Can we interpret latent reasoning using current mechanistic interpretability tools? *Alignment Forum*, December 2025.

J. Dilgren and S. Wiegreffe. Are latent reasoning models easily interpretable? *arXiv:2604.04902*, 2026.

S. Hao et al. Training large language models to reason in a continuous latent space. *arXiv:2412.06769*, 2024.

G. Huang and L. Qi. When steering fails in latent reasoning: A latent-to-language transition gap. *arXiv*, 2026.

Jin, Yang, and Wang. Final checkpoints are not enough: Analyzing latent reasoning faithfulness along training trajectories. *arXiv:2607.06648*, 2026.

Li et al. Dynamics within latent chain-of-thought: An empirical study of causal structure. In *ICML*, 2026. arXiv:2602.08783.

Open problems in mechanistic interpretability. *arXiv:2501.16496*, 2025.

Decodability is not causality: Dissociating probe readouts from behavioral drivers via SAE decomposition. *arXiv:2609.18080*, 2026.

Ramjee et al. Ulterior motives: Detecting misaligned reasoning in continuous thought models. ICLR 2026 Workshop on Latent and Implicit Thinking. arXiv:2604.23460.

Z. Shen et al. CODI: Compressing chain-of-thought into continuous space via self-distillation. In *EMNLP*, 2025. arXiv:2502.21074.

X. Wei et al. SIM-CoT: Supervised implicit chain-of-thought. In *ICLR*, 2026. arXiv:2509.20317.